11 User-Centred Agile Development to Support Sensemaking

M. E. N. Begnum
NTNU Gjøvik

CONTENTS

A Brief Introduction to User-Centred Design 174
UCD Core Principles 174
UCD as a Process 175
UCD Methods and Techniques 176
Understanding: Discover Insights 177
Specifying: Define Needs 177
Designing: Explore Solutions 177
Evaluating: Testing and Adjusting 178
A Brief Introduction to Agile Development 178
Agile Core Principles 179
The Agile Process 179
User-Centred Agile Development (UCAD) 180
Rushing Problem Framing 180
Forgetting the Context 181
Workload Discrepancies and Flow Issues 181
Parallel Model 181
Satellite Model 183
The UScrum Model 183
Lean UX Model 183
User-Centred Agile Model 184
Final Reflections 186
References 188

User-centred design (UCD) is a fitting approach for supporting sensemaking, as the aim of UCD is to understand contextual user needs and use this as a basis to iteratively explore and design solutions. Similarly, agile software development uses iterations and rapid feedback to continuously explore, learn and improve user stories and their implementations. The user-centred and agile approaches represent mindsets of great influence in their respective fields of design and software development. In this chapter, we will briefly introduce the two approaches. We present five models that can be used

to combine agile and user-centred development and discuss three common challenges in user-centred agile development (UCAD). Key take aways from the chapter are:

- The importance of exploring, understanding and framing the problem, in order to:
  - Solve the right challenge.
  - Propose solutions that fit the context of use.
  - Make sure user needs drives ideation – not the love of technology.
- The importance of user involvement and contextual testing as a team effort to:
  - Enable interdisciplinary decision-making and collaboration.
  - Enable continuous learning and improvement based on feedback.
  - Create solutions that work in real-life scenarios and contexts.

A BRIEF INTRODUCTION TO USER-CENTRED DESIGN

The aim of UCD is to create solutions that fit the needs of the users, in their contexts of use. Therefore, an explicit understanding of contextual user needs is key (e.g. Forlizzi & Battarbee, 2004; Pruitt & Grudin, 2003; Sengers, Boehner, David, & Kaye, 2005; Vetere et al., 2005). Users – not technology – are in focus throughout the design process (Hartson & Pyla, 2012; Rubin & Chisnell, 2008). As sensemaking is dependent on cues from key systems, training, physical environments and social contexts of use, a user-centred approach is recommended for designing technology that is to assist human sensemaking.

UCD is used synonymously with human-centred design (HCD). However, some argue there are slight differences between these terms – for example, that swapping "human" for "user" emphasizes that we are talking about real people with emotions and psychological preferences – not some abstract "user", and that a number of stakeholders may be impacted instead focusing in on a selected user group. As UCD extends usability and includes a holistic user experience (UX), user-centred work is often referred to as "UX work".

UCD Core Principles

The core principles of UCD are (ISO, 2019):

1. The design is based upon an explicit understanding of users, tasks and environments
2. Users are involved throughout design and development
3. The design is driven and refined by user-centred evaluation
4. The process is iterative
5. The design addresses the whole UX
6. The team includes multidisciplinary skills and perspectives

Gould and Lewis (1985) underline how early and continuous focus on users, empirical measurement of usage and iterative design are core UCD traits. Still, there is a wide range in the degree of user contact in UCD processes – from thinking about the user via different

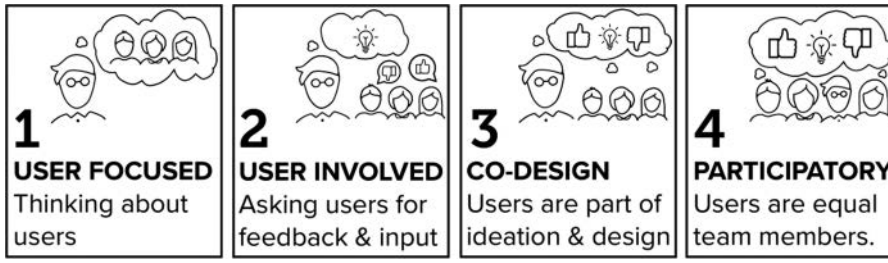


FIGURE 11.1 The range of user contact in user-centred design (UCD) approaches.

levels of empathy and involvement to users participating as team members. Different views on who should receive focus, the degree of direct contact and the user–designer relationship have separated user-centred methodologies into specific approaches, such as inclusive, empathic, co-creative and participatory design. For sensemaking, a mid- to high range of user involvement could probably be fitting, where the continuous involvement of users can provide an in-depth understanding of contextual needs.

Regardless of the chosen degree of direct contact, a situated understanding of user needs and pain points in order to ground the process is needed. Also, at least *some* direct user involvement is expected, particularly in user testing of prototypes, mock-ups and sketches.

UCD as a Process

The ISO 9241-210:2019 standard presents a generic process model iterating four phases: (1) understand and specify context of use, (2) specify user requirements, (3) produce design solutions to meet user needs and (4) evaluate (validate) designs (ISO, 2019). The ISO model emphasizes iterations but does not address the divergent (exploring) and convergent (selecting) stages that typically take place in a design process – unlike the double diamond model which emphasizes divergence/convergence over visualizing iterations back and forth.

The double diamond is commonly used by design disciplines and emphasizes the process of discovering needs and framing the design problem prior to exploring solutions (Tschimmel, 2012, p. 9). While direct user involvement takes time and effort, relying on indirect user contact runs the risk of creating a product no one needs. The same risk occurs if not enough time is allocated to ensure that the right problem is framed. Using the double diamond in early phases may help scoping and innovating problem definition and framing, including truly understanding the situated context of use. In principle, there is no inherent incompatibility between the double diamond and ISO models, and both are user centred. Their core phases are the same: understand/discover, specify/define, design and evaluate/deliver. The ISO model emphasizes iterations but does not address the divergent (exploring) and convergent (selecting) stages that typically take place in a design process – unlike the double diamond model which emphasizes divergence/convergence over visualizing iterations back and forth.

In UCD efforts, professionals may move between classic stances of positivist, critical and constructivist paradigms. The positivist stereotype seeks information

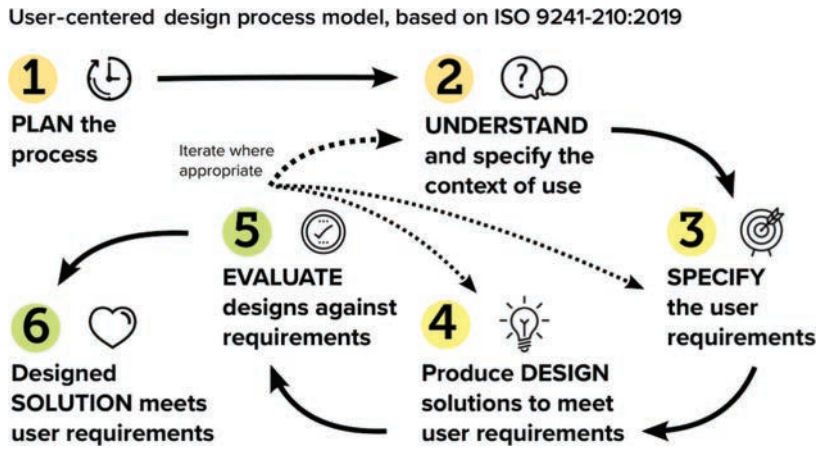


FIGURE 11.2 The user-centred design process.

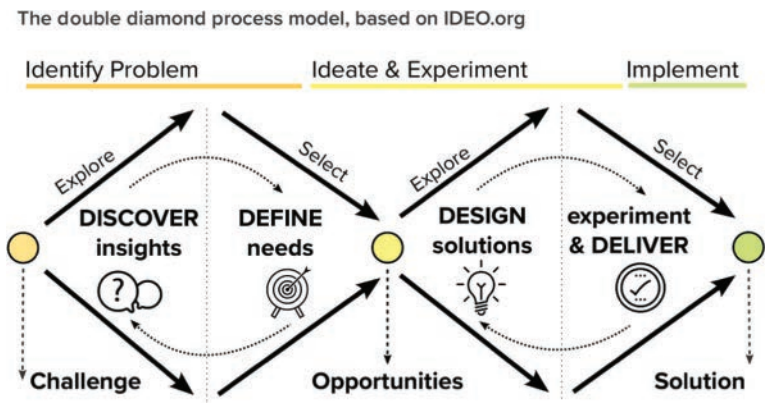


FIGURE 11.3 The double diamond model.

in order to advice: *What is known? How can we reach our aim?* The critical stereotype is advocating for positive change and triggering innovation by questioning: *Is this the right aim? Why?* The constructivist stereotype is a negotiator, aiming for shared insights and efforts: *Who are stakeholders and priorities? How can we collaborate?* Early on, one might feel the need to be more critical and exploratory in order to better frame the problem. There is not a single right way, model or approach to do UCD.

UCD Methods and Techniques

There is a wide and ever-growing range of UCD methods, and it takes experience and reflection to choose the "right" one at each step. The next sections outline some

of the most common methods, grouped to correspond to the core UCD phases: (1) understanding, (2) specifying, (3) designing and (4) evaluating. The iterative nature of the UCD process supports the selection of *any method* at *any time*, asking, e.g., What are the most important insights to gain next? How much time and resources do we have for UX work now and later?

Understanding: Discover Insights

Design processes typically start with an exploration of the problem area at hand. A core aspect of UCD is framing the goals in situated user needs. Thus, entering into the context of use to gather rich qualitative insights is recommended. UCD has traditionally been quite task focused on specific usage goals (e.g. Pinelle & Gutwin, 2003), overlapping with UCD. However, UCD also overlaps with anthropology in its field work to discover situated needs and the characteristics of the users. Key questions may be: *Who are our users and their experienced reality, needs and mental models? What is the context and nature of use?* This work is often called "user research" or "design research". You ask, listen and take notes.

Perhaps the most frequently used methods are interviews and observations (Begnum & Thorkildsen, 2015). These methods are flexible and may be contextual, formal or informal, spontaneous or planned, open and involved or closed and hidden. They include both focus groups and task analysis. There are many other methods that can also be utilized, such as workshops, card sort, probes, environmental analysis, empathic modelling and service safaris. Using quantified data from surveys or web analysis is a low contact method for knowledge gathering. Since they seldom help you identify pain points in complex real-life scenarios, they are not recommended as sole methods for finding and framing the right problem to solve.

Specifying: Define Needs

As UCD is iterative, the lines between gathering insights and specifying needs are blurred. You discover something, create assumptions, then go back to investigate interesting findings. Commonly used methods for specifying needs in UCD are affinity maps, personas, value propositions and scenarios-related techniques such as storyboards, user journeys and user stories. Many specifying methods visualizes the elicited insights. Such visual artefacts are recommended to convey user needs to stakeholders and facilitate team communication (Garcia, Silva da Silva, & Silveira, 2017). The visualizations can be updated as new discoveries are made. The specification phase is ended when you have framed your problem and feel certain that you have discovered true contextual needs and pain points.

Designing: Explore Solutions

Moving on to exploring solutions, there is a massive range of techniques available to ideate, create and evaluate design concepts – including blue-skying, brainstorming and bodystorming (D. Gray, Brown, & Macanufo, 2010). Many of the scenario-related techniques cross over to the concept exploration phase. User involvement and co-creation may be utilized to explore alternative design directions early on. They also help uncover user perspectives and additional insights into the underlying emotions and values behind user preferences. Workshops are recommended

for interdisciplinary discussions. In addition, role playing and physical prototypes are useful techniques. In order to express design concepts, low-fi prototypes such as sketches, mock-ups, presentations and whiteboarding are commonly used (C. M. Gray, 2016).

Evaluating: Testing and Adjusting

At some point, a selection of one or more design directions is made – for example, through prioritization matrices. Early on, focus should be on validating the value of a concept not on the usability of the design. It is not uncommon to involve users and stakeholders in design walkthroughs (called pluralistic walkthroughs). Increasingly, participatory and co-creative design work is also being used, enabling rapid user feedback and co-discovery tests.

Next, potential solutions are typically tested with users and adjusted in an iterative manner. An iterative process describes a strategy where something is produced, then evaluated and next refined. Low-contact methods are usually utilized as a first step, such as design critique based on design guidelines, heuristic evaluations and viability analyses, before moving on to user testing. Though formal (summative) testing is reportedly used in UCD literature, informal usability tests appear the most commonplace in industry. Informal usability testes are conducted as soon as practical – for example, through high-fi prototype guerrilla testing (going to public libraries, subway testing, asking cantina personal, taxi drivers, etc. depending on the user environment). In UCD, usability testing usually refers to user testing. Pilot user feedback is also utilized, as are online testing and online prompting for feedback (analytics tools). Pilot user feedback is also utilized, as are online testing and online prompting for feedback (analytics tools).

A BRIEF INTRODUCTION TO AGILE DEVELOPMENT

The term “agile” is applied to software development processes that are iterative and incremental, focused on continuous improvement of working code while leaving room for exploration and change (Petersen & Wohlin, 2010). An incremental process develops a larger system by partial deliveries. By delivering partial value continuously, two things happen: (1) it is possible to get early feedback and find what works and does not work for the user, with less time spent on planning and documentation and (2) through continuous testing and deployment pipelines, costumers and end-users get continuous value output and software quality is constantly improved.

Compared to traditional plan-based and overly requirements-based approaches, the agile mindset is more efficient in quickly and reliably producing a usable system (Beyer, 2010, p. 1). Early partial deliveries are usually not working code (e.g. prototypes). As the codebase matures, changes become smaller, allowing for an efficient automated deployment of the new code known today as continuous delivery or continuous deployment. Incremental deployments may include operations-related updates and roll backs, development-related features, bug fixes, redesigns and re-factoring. The agile mindset facilitates a high change tolerance from one iteration to the next, supports rapid exploration of technical solutions, priority adjustments based on feedback and continuous error fixing (Constantine, 2001).

The agile development aims of efficiency and reduced overhead overlaps with principles of lean development (as presented in, e.g., Hines, Holweg, & Rich, 2004). Some view lean as part of the agile mindset while others argue lean is a separate mindset more focused on production flow. Agile and lean share focus on continuous improvement as well but with different perspectives; agile methods focus on software development while lean is focused on product development in general. In this section, we discuss combining agile and UCD with focus on software development.

Agile Core Principles

The Agile Manifesto of 2001 declared 12 principles, summarized as: (1) satisfy the customer, (2) welcome changed requirements, (3) deliver software frequently, (4) involve businesspeople daily, (5) trust team members and ensure their motivation, (6) use face-to-face communication, (7) measure progress in working software, (8) keep a sustainable pace, (9) continuous attention to quality, (10) keep it simple, (11) use self-organized teams and (12) do regular effectiveness reflections (Beck et al., 2001). Overarching these were four values: individuals over process, working software over unnecessary documentation, change response over following a plan and customer collaboration over contract negotiations. Since then, the agile values have been updated to the following modern agile principles (Modern Agile):

1. "Make People Awesome"
2. "Make Safety a Prerequisite"
3. "Experiment & Learn Rapidly"
4. "Deliver Value Continuously"

The focus of the Modern Agile community is aiding costumers to be great at what they do, no longer simply on creating a great product (ModernAgile). As such, the separation between the costumer and the team (as a supplier) is minimized over time. Earlier, agile development was ended when the costumer was satisfied with the supplied solution. Today, one recognizes delivery is continuous throughout the lifecycle of a solution. This has inspired DevOps practices of merging development and IT operations by deploying features into production quickly and continuously correct problems. A modern agile process is ended when the decision is made to discard or redevelop a solution.

The Agile Process

There are different ways of organizing the agile teamwork to support the agile principles, and as such there are infinite variants of the agile processes. Some popular agile frameworks are Extreme Programming (XP), Scrum and Kanban (Agile Alliance):

- Scrum is a widely known framework. The Scrum model is focused on facilitating self-organizing short push-iterations (sprints) – including efficiency reviews and team roles. The team delivers new software to the costumer at certain time intervals.

- XP is the most specific framework, pushing specific methods and practices such as co-location, pair programming, weekly iterations, continuous integration and quarterly releases.
- Kanban is regarded today as agile but has roots in lean thinking of eliminating bottlenecks and moving from push- to pull-based flows. Kanban does not set time-limited iterations (sprints) and instead focuses on limiting the number of work tasks in progress.

It is common for companies and teams to create their own process models by adjusting the proposed agile frameworks. Overall, iterative feedback loops, rapid learning and incremental software delivery are key agile attributes – as is the role of the team. Agile teams are usually multidisciplinary, and team members organize tasks and work amount in a democratic manner. Though many agile teams apply time-limited iterations that ends with a delivery (Scrum-like), this is not a must. Some agile teams strive to achieve efficient workflows (Kanban-like). Currently, there appears to be a shift from the early agile process approaches towards even lighter frameworks guided by the modern agile principles – without elevating specific practices.

As agile has evolved, focus on risk management and user empathy has also been increased. Modern agile approaches seem less focused on estimating story points for feature development and the efficiency (velocity) of the team to develop features. Instead, the importance of a safe culture for experimenting and learning from mistakes is emphasized. As such, even if efforts are planned with the aim of incremental delivery, there is a shift towards learning of end-user needs through product development iterations. Note that just as in UCD, agile processes are not mainly focused on initial problem framing activities, but rather on experimentation based on assumptions of needs.

USER-CENTRED AGILE DEVELOPMENT (UCAD)

Both UCD and agile mindsets embrace iterative and incremental processes with frequent feedback loops to explore, learn and improve. These practices have proved valuable in delivering the right solutions to the problems at hand. However, there are indications that some user-centred values are overlooked in the agile world. The next sections discuss the three main dangers when merging UCD and agile development: (1) rushing problem framing, (2) interface without context and (3) workload discrepancies and flow issues.

Rushing Problem Framing

User-centred methods are well suited for gathering in-depth insights, identifying needs and scoping the design space based on user needs – including deciding which needs are the most critical to solve and innovating on possible strategies to do so. Though well suited for problem solving, agile settings have less tools for problem framing – including exploring whether software development is the right approach to solve a user need. Compared to the user-centred double diamond model, agile frameworks do not emphasize the early process of problem framing prior to exploring solutions. Experimenting in incremental and iterative development is not the same

as problem framing. Singh (2008) notes that agile product goals are sometimes set without ensuring an adequate study of the user's needs and context.

User research and problem framing can be embedded into UCAD. However, many UCAD processes do not emphasize framing or the iterative, divergent and convergent aspects of this phase. UX designers may be expected to quickly gather user insights, identify contextual needs and frame the problem on their own, once and at the start of the process. If the team pushes to start coding, this necessitates the problem at hand is already well defined. If not, the problem at hand may be based on weak or faulty assumptions about users and context of use (Constantine, 2001) – which will also make the solutions to the problem ill-suited for real-life scenarios.

Forgetting the Context

Usage is affected by the social, physical and emotional contexts, by noise, time and space, and by what happens prior to and parallel to any system interactions. These aspects must be taken into consideration both when framing and understanding the problem at hand and in the design and evaluation of the potential solutions. Prototype testing, even if simulating realistic scenarios, may not be sufficient on its own to elicit needs.

It appears UCAD settings have less user contact and utilization of user involvement compared to traditional UCD, emphasize user interface (UI) over other design activities and prioritize feature implementation over in-depth user research (Begnum & Thorkildsen, 2015; Silva da Silva, Silveira & Maurer, 2013). Singh (2008) observes that agile development teams sometimes have difficulties seeing the whole UX and relevant user-centred aspects, and instead focus on just-in-time deliveries. In a worst-case scenario, one may imagine a team holding a faulty understanding of the real issue at hand, a weak understanding of contextual needs and testing solutions in non-realistic settings – such as a laboratory.

Focus on UI without investigating real-life issues may lead to the team being unaware of important contextual aspects influencing the overall UX and usability of a product. This is particularly important with regard to safety-critical solutions used in complex sensemaking settings, where disregarding social, emotional, contextual and timely aspects (e.g. non-verbal communications, expectations, stress, noise, distractions, fatigue) may carry huge risks.

Workload Discrepancies and Flow Issues

UCAD also faces challenges related to workload discrepancies, interdisciplinary collaboration and flow issues. At least five generic models have been suggested for UCAD, aiming to mitigate some of the issues described in the previous sections. The models are not necessarily mutually exclusive nor are they exhaustive on possible ways to organize UCAD workflows.

Parallel Model

In the *parallel model*, UX work and coding is separated, forming two different "tracks". It has also been referred to as the "dual track agile" model and proposes doing UX work ahead of development (Miller, 2005; Sy, 2007).

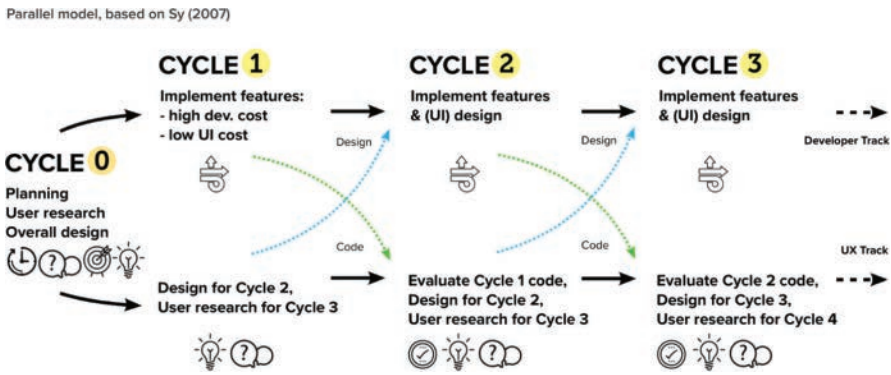


FIGURE 11.4 The parallel model.

The first thing to note is that though the model starts with a Cycle 0, encompassing user research, problem exploration and framing are not emphasized in the parallel model. Cycle 0 focuses on planning how to build an already agreed-upon problem and includes ensuring sufficient insights into contextual user needs is held by the team. If a UX designer starts questioning the planned solutions, this is an indication you have rushed the problem framing.

The second thing to note is that the parallel model does not remind the team to remember contexts of use. In fact, the UX track was originally labelled the "Interaction Designer track" by Sy (2007) and focused on UI design. With a UI focus, Cycle 0 is likely to be used as an iteration for planning the overall UI design of an agreed-upon concept.

The third thing to note is that the team must plan several sprints ahead in this model and as such is less adaptable to change. Any changes could necessitate rework for the UX track. The UX track is at last one sprint ahead of the coding track, conducting contextual inquiries, prototyping and user testing prior to passing on validated designs ready for implementation to the development track. If the coded designs need further user testing, this would also be UX work, and as such a designer would work both ahead of and behind the developers. This has been described as exhausting by designers (anecdotal evidence, unpublished interviews).

Finally, the separation of UX and development split the team – with developers often leaving all the UX work to one designer. Thus, two parallel processes go on at the same time, with iterating deliveries from one track to the other. The interaction between these two tracks is structured and frequent, but if separated into developers waiting for UX input, there is a risk of "UX bottlenecks" hindering workflow due to workload discrepancy. Also, parallel work organization hinders interdisciplinary discussions and problem solving. Parallel models thus appear the most advantageous when there are adequate resources to secure collaborative team efforts within a sustainably paced work environment, with the team pulling together on both tracks and where there is UCD competence (e.g. with a designer guiding team UX work).

Satellite Model

The idea of the *Satellite model* is to solve the issue of a single UX resource per development team becoming exhausted or a bottleneck. Here, one satellite UX person is co-located with an agile team as a UX specialist, focused on supporting the agile development. The UX specialist also facilitates collaboration with the full UX team (Kollmann, Sharp, & Blandford, 2009). The UX team does ideation, user involvement, prototyping and usability testing on several projects, as a joint effort. The UX team may pull backlog items directly, disconnected from development, and may work on different features. Apart from the UX specialist, the UX team is separate from the development team in this model (Øvad, 2014). One worry is that incorrect assumptions may be made by the UX team or the developer team. As such, the model improves work balance and flow between development and UX – but does not create opportunities for interdisciplinary collaboration.

The UScrum Model

Singh (2008) proposed the *UScrum model* where an UX management role is added to Scrum, in order to solve the challenges of problem framing and understanding contextual needs as well as ensuring user-centred tasks for ensuring that usability is not de-prioritized. To do this, UScrum splits the Scrum product owner role in two where one takes on an explicit usability focus and the other focuses on feature implementation. With regard to modern agile thinking, the idea that a product owner is feature-focused feels outdated. A product owner is expected to provide the team with insights into business *and* user needs and help with work prioritizing. Furthermore, the traditional agile focus on making the customer *happy* is changed to *awesome* in modern agile – increasing end-user focus. Nonetheless, UScrum represents a way to improve work balance and flow between development and user-centred perspectives without splitting into two teams or tracks, thus supporting interdisciplinary collaboration.

Lean UX Model

A newer model for integrating UCD and agile mindsets is *lean UX*. This model relies on modern agile values and aims to support moving to iterations based on *assumptions* instead of creating an established to-do backlog with user stories (Øvad, 2014).

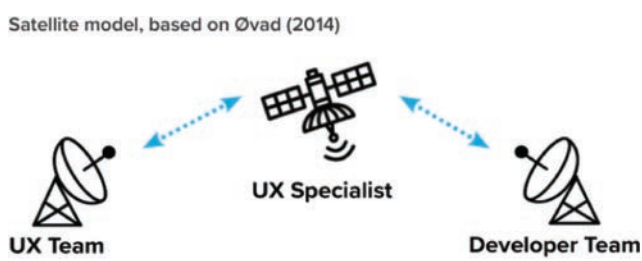


FIGURE 11.5 The satellite model.

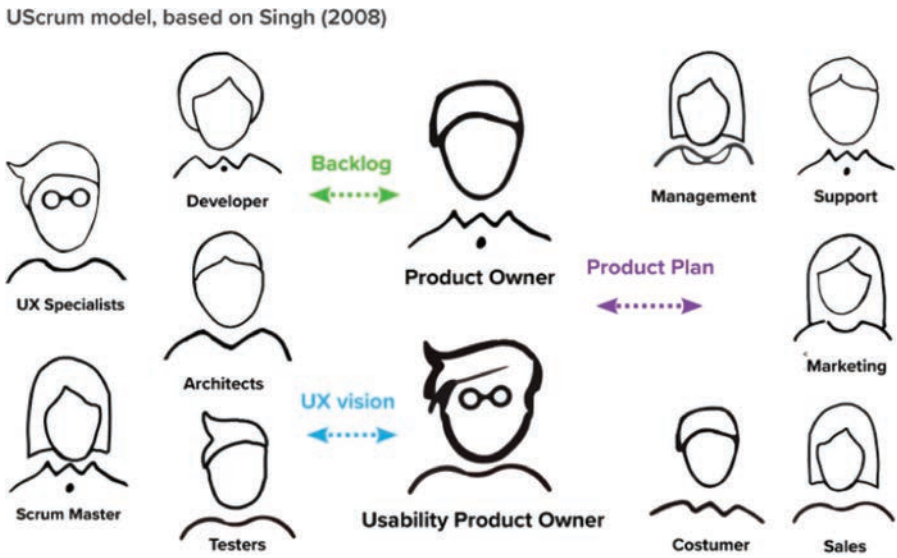


FIGURE 11.6 The UScrum model.

Lean UX proposes four steps: (1) research, (2) declare assumptions, (3) create a solution (based on the assumptions), and (4) test (measure) the solution (and assumptions) through an experiment. The shift from stating a requirement to declaring an assumption clarifies the need for contextual experimentation. The model thus aligns well with the UCD model and communicates better how decision points are constantly needed for clarification on the viability of a solution. The model emphasizes the mindset of experimenting and learning, rather than the incremental delivery, and facilitates interdisciplinary collaboration.

User-Centred Agile Model

The user-centred agile model combines an initial user-centred process to help understand the user and context, discover needs and frame the problem with agile development to iteratively and incrementally experiment, build, learn, deploy and improve software. The user-centred agile model uses the double diamond model for initial UCD and the lean UX model for agile development, and as such combines UCD, design thinking, lean and agile approaches.

This model encompasses both initial and continuous user-centred explorations, focusing both on building the right thing and building the thing right. Compared to the parallel model, user research and user involvement is now explicitly integrated as part of the UCAD process. This combats the reduction of UCD to UI design and the subsequent omittance of physical, social and emotional contexts of use and sensemaking. Furthermore, the user-centred agile model prevents problem framing from being skipped. Implicit in the model is the belief that the team only moves into lean UX if they believe that software development can solve the problem at hand.

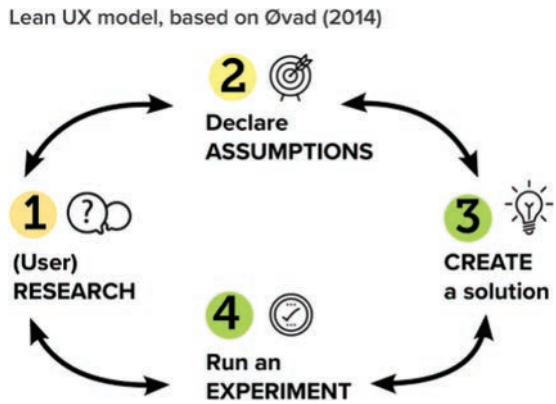


FIGURE 11.7 The lean user experience (UX) model.

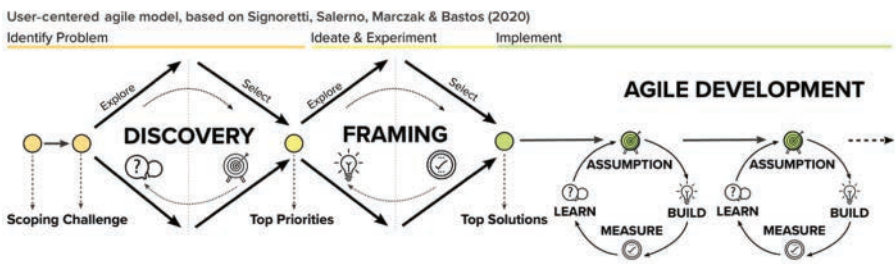


FIGURE 11.8 The user-centred agile model.

A recent case study documented the effect of implementing the user-centred agile model and showed that by involving the whole team in framing stages (such as user interviews, user research and problem exploration), there was a shift from focusing on requirements to map user and business problems (Signoretti, Salerno, Marczak, & Bastos, 2020). This promoted a problem-oriented mindset and shared ownership. This mindset merged well with the assumption-based lean UX iterations, as teams reported they felt safer to fail.

Another finding in their work was that the overall user focus was strengthened. The individual team members reported feeling responsible for investigating the problem at hand and actively seeking to understand user needs. Further, this shared UX responsibility resulted in increased cross-disciplinary decision-making processes, and it particularly influenced the role and collaborative attitude of the developers. Overall, teams reported increased engagement, user empathy and a shared project vision (Signoretti, Salerno, Marczak, & Bastos, 2020).

As such, preliminary research indicates the user-centred agile model is successful in supporting UCAD teams in strengthening problem-oriented mindsets, collaborative decision-making and user empathy. Both from theoretical assumptions and

empirical evidence the model shows promise in relation to alleviating the three main UCAD challenges reported by research: (1) rushed problem framing, (2) forgetting the context and (3) workload discrepancies and flow issues.

Final Reflections

Neither UCD nor agile presents a one-size-fits-all best-practice approach. Instead, one draws on mindsets, methodologies and process models and adapts these to the problem, the context and the available resources at hand. An approach fitting in one setting does not necessarily fit the next. The same is the case for UCAD. As long as you adhere to the mindsets of both UCD and agile development, you can safely draw on a wide range of practices and define workflows that suit your needs while remaining certain that you are creating solutions that work for your users in real life and real context of use.

Literature reports a culture gap between software designers and developers, where diverging core values create challenges in collaboration and communication (e.g. Bhrel, Meth, Maedche & Werder, 2015; Beyer, Holtzblatt & Baker, 2004; Salah, Paige & Cairns, 2014). Bhrel, Meth, Maedche and Werder (2015) present an extensive literature survey on UCAD including 84 research papers in order to define a common set of UCAD principles. They are nonetheless unable to derive any principles related to team organization, collaboration, communication, decision-making, roles and responsibilities. Their conclusion is that literature does not present a clear alternative to teamwork organization as parallel tracks, even though there is very limited empirical evidence supporting the one sprint ahead concept. Moreover, they note that research does not point out two separate teams in parallel as being the optimal. About 41% of the included papers described cross-disciplinary agile teams – and not two dedicated teams handling UX and development. They paraphrase Kuusined (2012) who reports that the most frequent advice from practitioners on improving cooperation, decision-making and knowledge transfers is having UX competence available within the team (2014). Salah, Paige and Cairns (2014) surveyed 71 papers and recommend (1) sharing an understanding of users, (2) sharing an understanding of design vision, and (3) synchronization of parallel efforts to allow for collaboration (e.g. daily synchronization points) in order to optimize workflows and cross-disciplinary relationships.

On the topic of building the right product vs. building the product right, Bhrel, Meth, Maedche and Werder (2015) conclude that research supports a principle of separating product discovery and product creation. They reference Kettunen (2009) who proposed drawing on UCD to counter agile shortcomings with regard to large-scale product innovations, scoping or ideation. However, they do not specify whether the agile team should be involved in both product discovery and product creation or whether they envision a handoff from discovery to the agile team. Beyer, Holtzblatt and Baker (2004) recommend that agile teams use rapid contextual design (rapid CD) to support problem framing and design and present a stepwise approach with specified methods that appear detailed, but otherwise align with UCD and the double diamond model. However, they reduce UX work to UI design and envision the skillsets of design and development remain separated – similarly organized as in

the parallel model. They propose costumer(s) and designer(s) complete agile rapid CD, subsequently handing off completed UIs and user stories to development. From the extensiveness of the rapid CD process, workflow bottlenecks are a risk and developers would not be involved in decision-making and design choices based on their process descriptions.

It is interesting that any disciplinary culture gap seems to be bridged by applying the user-centred agile model, where Signoretti, Salerno, Marczak and Bastos (2020) documented that both user focus and problem orientation were increased and that the UX responsibility and decision-making was shared. We now have an alternative to the parallel model for teamwork organization that fits better with recommendations from Salah, Paige and Cairns (2014). It is also worth noting that the gap identified in research appears to decrease over time, as modern agile and UCD principles and practices become better aligned.

Efforts are ongoing on how to integrate UCD into agile settings; however, these quickly appear dated. Alternative visualizations related to the user-centred agile model can be found on online blogs from around 2018, where design thinking, lean and agile frameworks are combined. However, most of them are focused on merging lean UX with agile and not including UCD specifically. Though the blog posts hold very interesting reflections, they so far appear only anecdotal from the writer's point of view and without scientific publications demonstrating their promise.

Based on the evidence at hand, we would generally caution against parallel tracks and advise to move towards more integrated teams where both UX work and code work are shared by the whole team, in order to ensure better collaboration and flow. Parallel workflows, proposed by previous research on integration models, may be counterproductive to efficient multidisciplinary team collaboration due to the separation of UX/UI design and development decisions. Furthermore, the risk for workload discrepancies may lead to ineffectiveness, communication issues and undermined design decisions. A parallel workflow setup needs generous constraints to ensure a smooth flow and cooperation between iterative UI design, development and UX work.

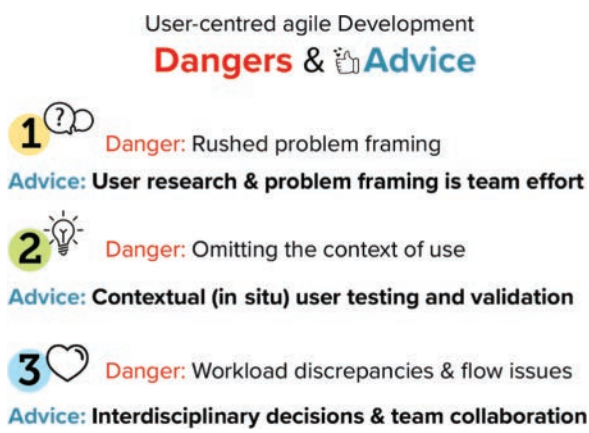


FIGURE 11.9 Summary: current dangers and advice.

Instead, a closer and continuous cross-disciplinary collaboration is suggested, focusing on a productive team-driven workflow and interdisciplinary decision-making. Problem framing and user research are recommended as *team* efforts. User testing and contextual (in-situ) design validation are recommended as *team* activities. A negative consequence of downplaying UX design is that the *team* gets a weaker understanding of the problem to be solved and the real user needs in real-life scenarios – which is especially dangerous with regard to safety critical systems and complex sensemaking settings.

REFERENCES

Agile Alliance. *Agile Essentials: Agile 101*. https://www.agilealliance.org/agile101/

Beck, K., Beedle, M., Bennekum, A. V., Cockburn, A., Cunningham, W., Fowler, M., … Thomas, D. (2001). *Manifesto for Agile Software Development*. http://AgileManifesto.org (accessed 21.02.2021); https://moodle2019-20.ua.es/moodle/pluginfile.php/2213/mod_resource/content/2/agile-manifesto.pdf (accessed 21.02.2021).

Begnum, M. E. N., & Thorkildsen, T. (2015). Comparing User-Centred Practices in Agile Versus Non-Agile Development. *Paper presented at the Norsk konferanse for organisasjoners bruk av IT (NOKOBIT)*, Ålesund, Norway.

Beyer, H. (2010). *User-Centered Agile Methods*. Morgan & Claypool.

Beyer, H., Holtzblatt, H., & Baker, L. (2004). An Agile Customer-Centered Method: Rapid Contextual Design. In Zannier E. H., Lindstrom L. (Ed.), *Extreme Programming and Agile Methods - XP/Agile Universe 2004*, LNCS vol. 3134, 50–59. Springer.

Bhrel, M., Meth, H., Maedche, A., & Werder, K. (2015). Exploring principles of user-centered agile software development. *Information and Software Technology, 61*(C), 163–181.

Constantine, L. L. (2001). Process agility and software usability – towards lightweight usage centred design. *Information Age, 8*(8), 1–10.

Forlizzi, J., & Battarbee, K. (2004, August 1–4, 2004). Understanding Experience in Interactive Systems. *Paper presented at the DIS2004*, Cambridge, MA.

Garcia, A., Silva da Silva, T., & Silveira, M. S. (2017). Artifacts for Agile User-Centered Design: A Systematic Mapping. *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS)*.

Gould, J. D., & Lewis, C. (1985). Designing for usability: key principles and what designers think. *Communications of the ACM, 28*(3), 300–311.

Gray, C. M. (2016, May 7–12). It's more of a Mindset Than a Method: UX Practitioners' Conception of Design Methods. *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, San Jose, CA.

Gray, D., Brown, S., & Macanufo, J. (2010). *Gamestorming: A Playbook for Innovators, Rulebreakers, and Changemakers*. Sebastopol, CA: O'Reilly.

Hartson, R., & Pyla, P. (2012). *The UX Book; Process and Guidelines for Ensuring a Quality User Experience*. Waltham, MA: Elsevier.

Hines, P., Holweg, M., & Rich, N. (2004). Learning to evolve: a review of contemporary lean thinking. *International Journal of Operations & Production Management, 24*(10), 994–1011. doi:10.1108/01443570410558049

ISO. (2019). ISO 9241-210:2019 Ergonomics of human-system interaction. In *Part 210: Human-centred design for interactive systems*. www.iso.org

Kollmann, J., Sharp, H., & Blandford, A. (2009). The Importance of Identity and Vision to User Experience Designers on Agile Projects. *2009 Agile Conference*.

Miller, L. (2005). Case Study of Costumer Input for a Successful Product. *Paper presented at the Proceedings of Agile Conference (AGILE)*, Denver, CO.

Modern Agile. http://modernagile.org/.

Øvad, T. (2014). *The Current State of Agile UX in the Danish Industry: The Analysis*. p. 33. https://vbn.aau.dk/ws/portalfiles/portal/209798499/The_Current_State_of_Agile_UX_in_the_Danish_Industry.pdf (accessed 21.02.2021).

Petersen, K., & Wohlin, C. (2010). The effect of moving from a plan-driven to an incremental software development approach with agile practices, An industrial case study. *Empirical Software Engineering, 15*(6), 654–693. doi:10.1007/s10664-010-9136-6

Pinelle, D., & Gutwin, C. (2003). Task analysis for groupware usability evaluation: modeling shared-workspace tasks with the mechanics of collaboration. *ACM Transactions on Computer-Human Interaction, 10*(4), 281–311.

Pruitt, J., & Grudin, J. (2003). *Personas: Practice and Theory*. DUX '03: Proceedings of the 2003 conference on Designing for user experiences, in San Francisco California June 2003, pp. 1–15, https://doi.org/10.1145/997078.997089.

Rubin, J., & Chisnell, D. (2008). *Handbook of Usability Testing, Second edition: How to Plan, Design, and Conduct Effective Tests* (2nd ed.). Indianapolis, IN: Wily.

Salah, D., Paige, R., & Cairns, P. (2014). A Systematic Literature Review for Agile Development Processes and User Centred Design Integration. *18th International Conference on Evaluation and Assessment in Software Engineering (EASE '14)*, London, England.

Sengers, P., Boehner, K., David, S., & Kaye, J. J. (2005, August 21–25, 2005). Reflective Design. *CC '05-4th Decennial Conference on Critical Computing: Between Sense and Sensibility*. Århus, Denmark.

Signoretti, I., Salerno, L., Marczak, S., & Bastos, R. (2020). Combining user-centered design and lean startup with agile software development: a case study of two agile teams. In: Stray V., Hoda R., Paasivaara M., Kruchten P. (eds) *Agile Processes in Software Engineering and Extreme Programming. XP 2020*. Lecture Notes in Business Information Processing, vol 383. Cham: Springer.

Silva da Silva, T., Silveira, M. S., & Maurer, F. (2013). Ten Lessons Learned from Integrating Interaction Design and Agile Development. *Agile Conference (AGILE)*, Nashville, TN.

Singh, M. (2008). U-SCRUM: An Agile Methodology for Promoting Usability. *Agile 2008 Conference*.

Sy, D. (2007). Adapting usability investigations for agile user-centered design. *Journal of Usability Studies, 2*(3), 112–132.

Tschimmel, K. (2012). Design Thinking as an Effective Toolkit for Innovation. *The International Society for Professional Innovation Management (ISPIM)*, Manchester.

Vetere, F., Gibbs, M. R., Kjeldskov, J., Howard, S., Mueller, F. F., Pedell, S., … Bunyan, M. (2005, April 2–7, 2005). Mediating Intimacy: Designing Technologies to Support Strong-Tie Relationships. *Paper presented at the CHI 2005*, Portland, OR.